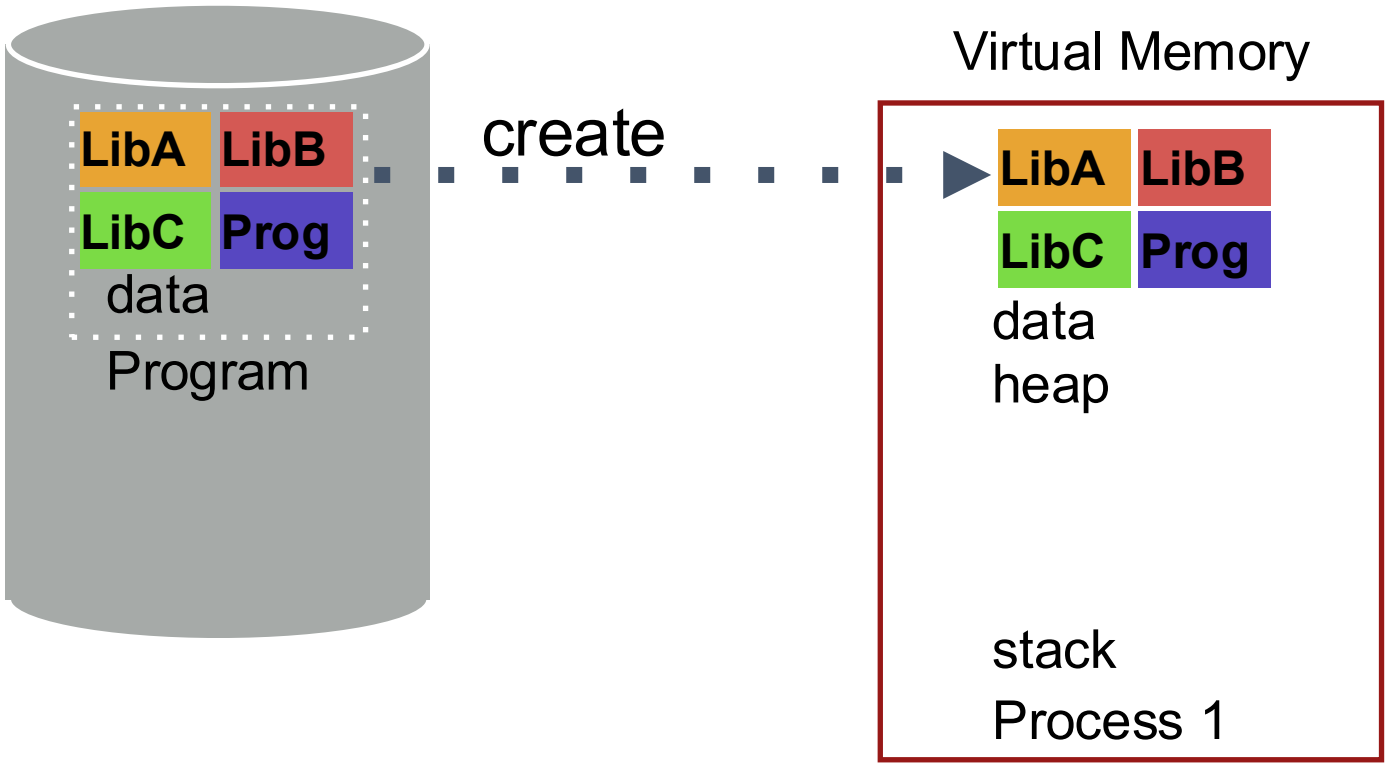
Memory Virtualization

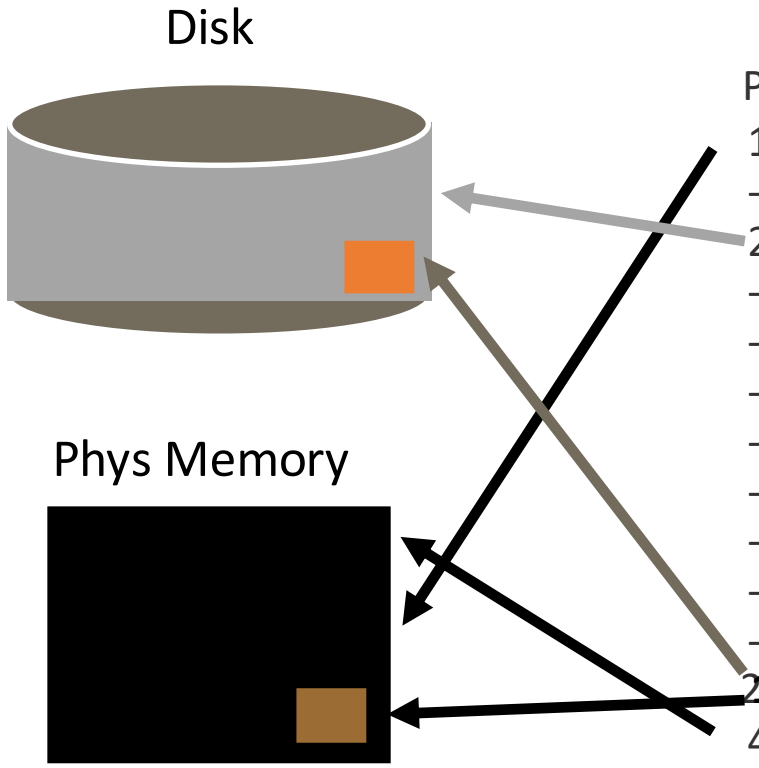




many large libraries, some
of which are rarely/never used

How to avoid wasting physical pages to back rarely used virtual pages?

Review: The “Present” Bit



PFN	valid	prot	present	
10	1	r-x	1	
-	0	-	-	
23	1	rw-	0	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
28	1	rw-	0	// VPN 0xb
4	1	rw-	1	

What if access vpn 0xb?

Review: Virtual Memory
Policies



Goal: Minimize number of page faults

- Page faults require milliseconds to handle (reading from disk)
- Implication: Plenty of time for OS to make good decision

OS has two decisions

- Page selection
  - **When** should a page (or pages) on disk be **brought into** memory?
- Page replacement
  - **Which r**esident page (or pages) in memory should be **moved out** to disk?

Review: Virtual memory policies



- Page selection:
  - Demand paging, pre-paging (prefetching), user-supplied hints
- Page replacement:
  - OPT: evict the page accessed furthest in the future
  - FIFO
- Compulsory misses and capacity misses

Least Recently Used (LRU)

Page reference string: 1,2,3,1,2,4,1,4,2,3,2

Three pages of physical memory



Metric: Miss count

5 total misses
4 compulsory misses

In this example, same as OPT.

Page Replacement Comparison

Add more physical memory, what happens to performance?

- LRU, OPT: Add more memory, guaranteed to have fewer (or same number of) page faults
  - Smaller memory sizes are guaranteed to contain a subset of larger memory sizes
  - Stack property: smaller cache a subset of bigger cache

- FIFO: Add more memory, usually have fewer page faults
  - "Belady's anomaly": there are cases where we have more page faults with more memory

Consider access stream: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Consider physical memory size: 3 pages vs. 4 pages

How many misses with FIFO?

3 pages: 9 misses
4 pages: 10 misses

Problems with LRU-based Replacement

LRU does not consider frequency of accesses

- Is a page accessed **once** in the past equal to one accessed **N** times?
- Common workload problem:
  - Scan (sequential read, never used again) one large data region flushes memory

Solution: Track frequency of accesses to page

Pure LFU (Least-frequently-used) replacement

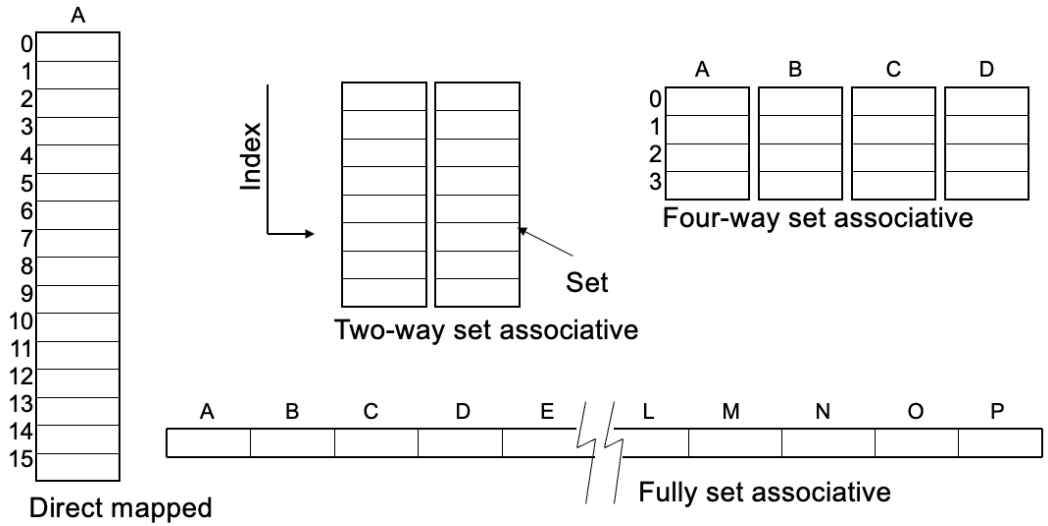
- A problem: LFU cannot forget pages from the distant past

State of the art: combine frequency and recency

Implementing LRU

Key challenge: act upon page fault, and also page reference!

We treat the physical memory to be fully associative
- Cost of a page fault is too high
- All physical frames are candidates for eviction



Implementing LRU

Key challenge: act upon page fault, and also page reference

Full LRU in Software

- OS maintains a list of physical pages ordered by reference time
- When page is referenced: Move page to front of list
- When need victim: Pick page at back of list
- Trade-off: Slow on memory reference, fast on replacement
- Software interposition on every memory reference

Full LRU in Hardware

- Associate timestamp with each page (e.g., add to PTE)
- When page is referenced: Associate current system timestamp with page
- When need victim: Scan through PTEs to find oldest timestamp
- Scanning long lists (e.g., large # physical frames) challenging in hardware, hence must do in software
- Trade-off: Fast on memory reference, slow on replacement

Implementing LRU

In practice, do not implement full, perfect LRU

- LRU approximates OPT, why not approximate more?
- Goal: Find an old page, but not necessarily the oldest

Clock Algorithm

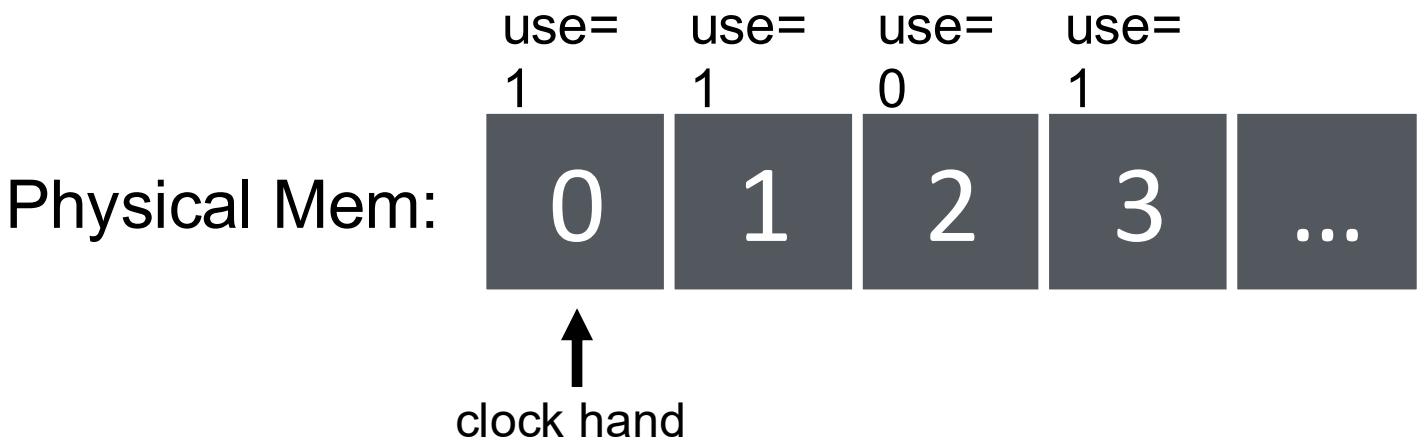
Hardware

- Keep `use` (or `reference`) bit for each page frame
- When page is referenced: hardware sets `use` bit

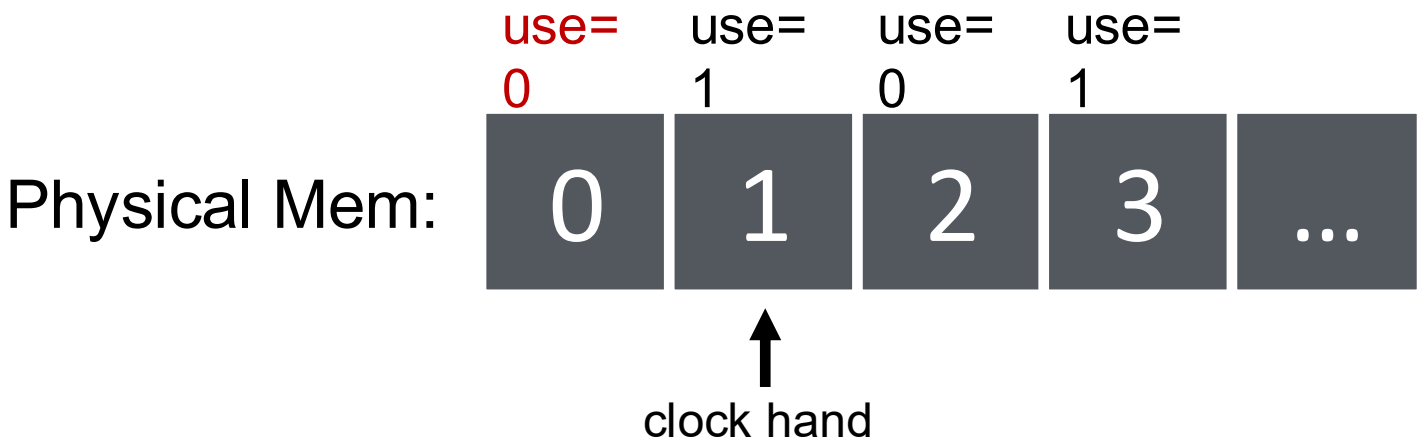
Operating System

- Page replacement: Look for page with `use` bit cleared (has not been referenced for a while)
- Implementation:
  - Keep pointer to last examined page frame (“clock hand”)
  - Traverse pages in circular fashion (like a clock)
  - Clear `use` bits as you search
  - Stop when find page with already cleared `use` bit, replace this page

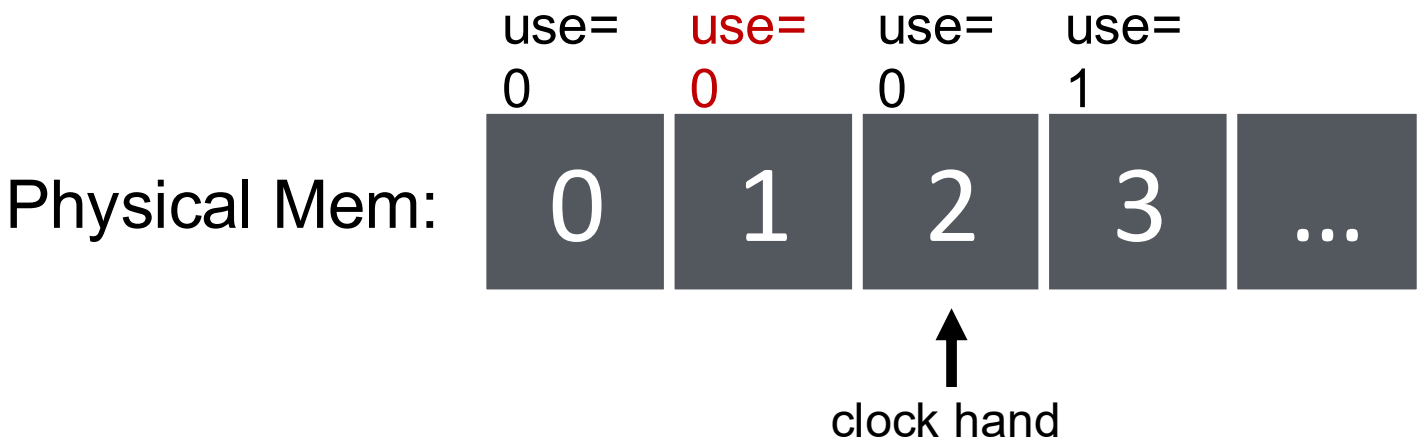
Clock:
Look For a Page



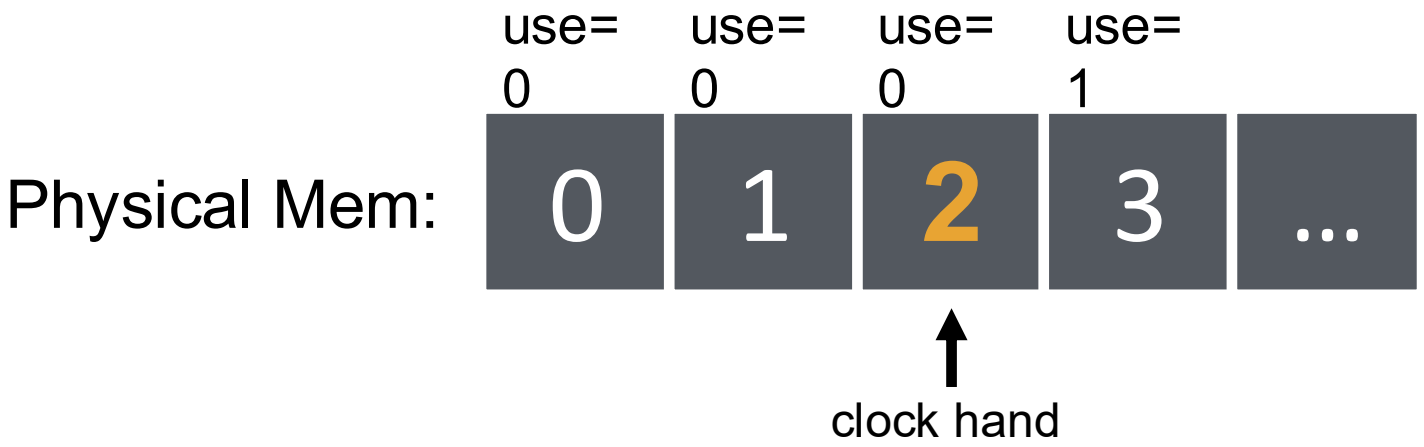
Clock:
Look For a Page



Clock:
Look For a Page

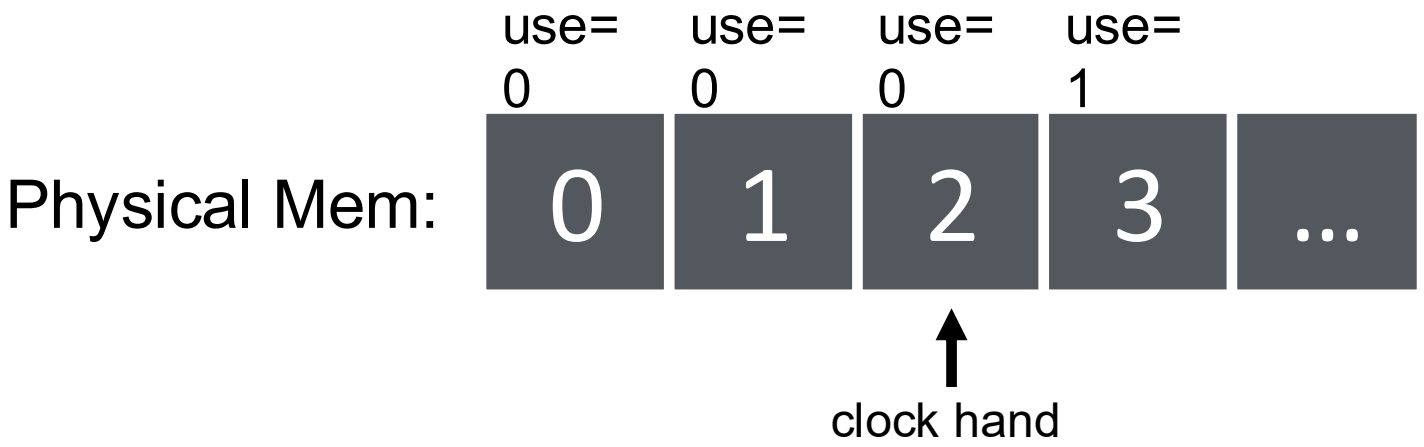


Clock:
Look For a Page



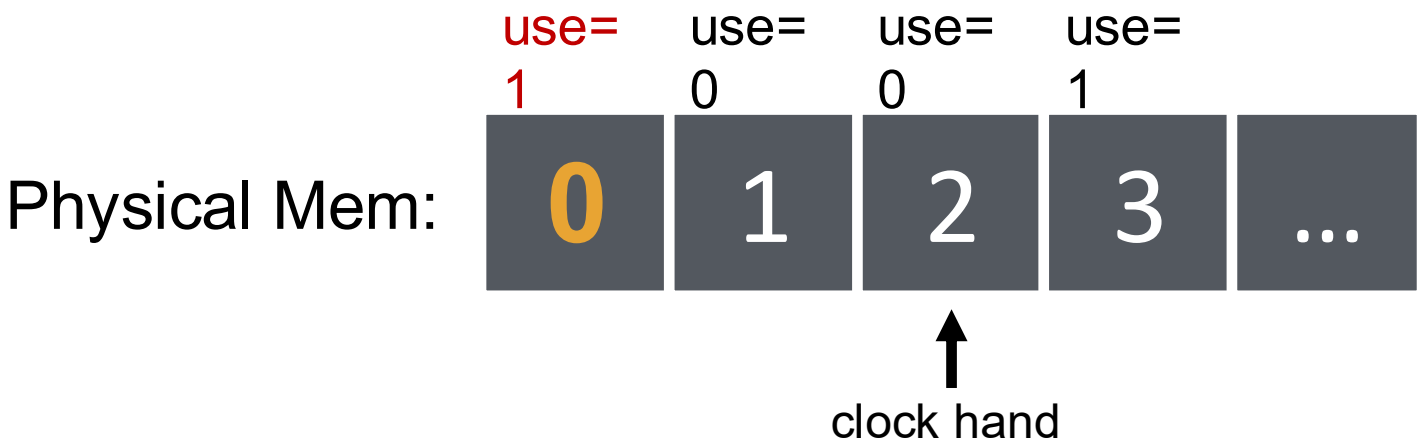
evict **page 2** because it has not been recently used

Clock:
Look For a Page

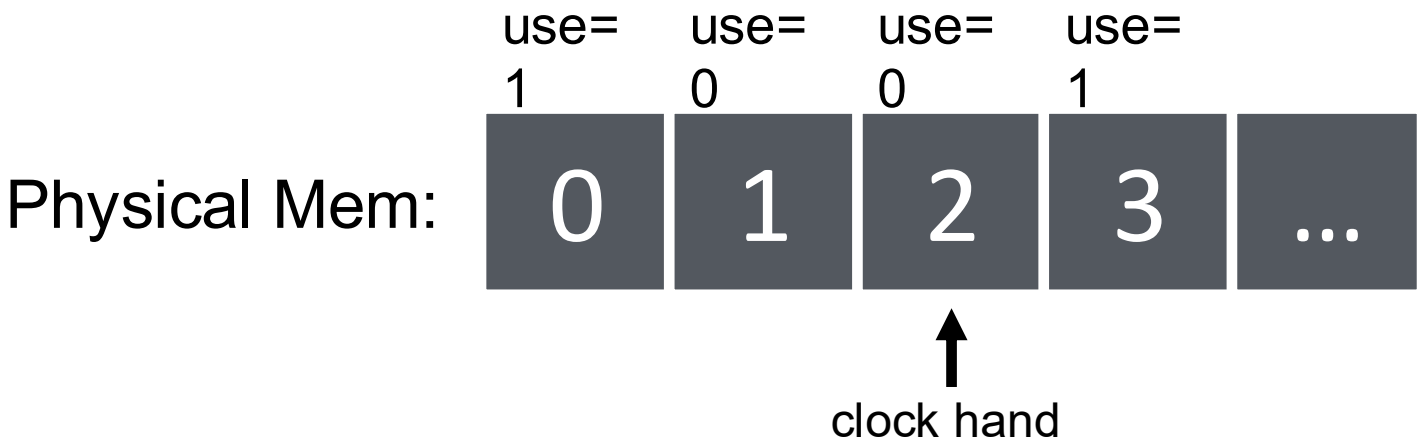


page 0 is accessed…

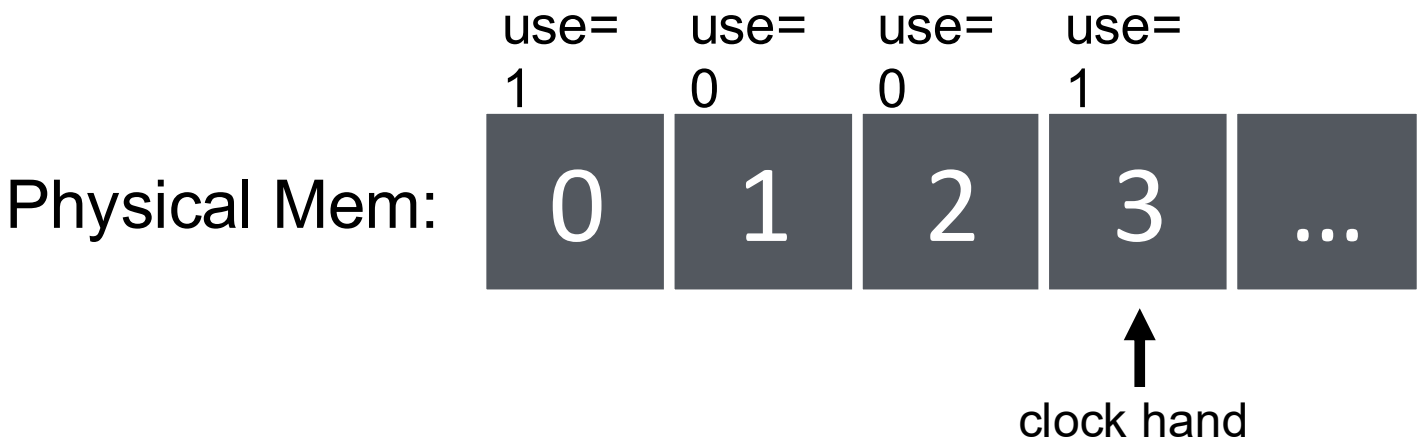
Clock:
Look For a Page



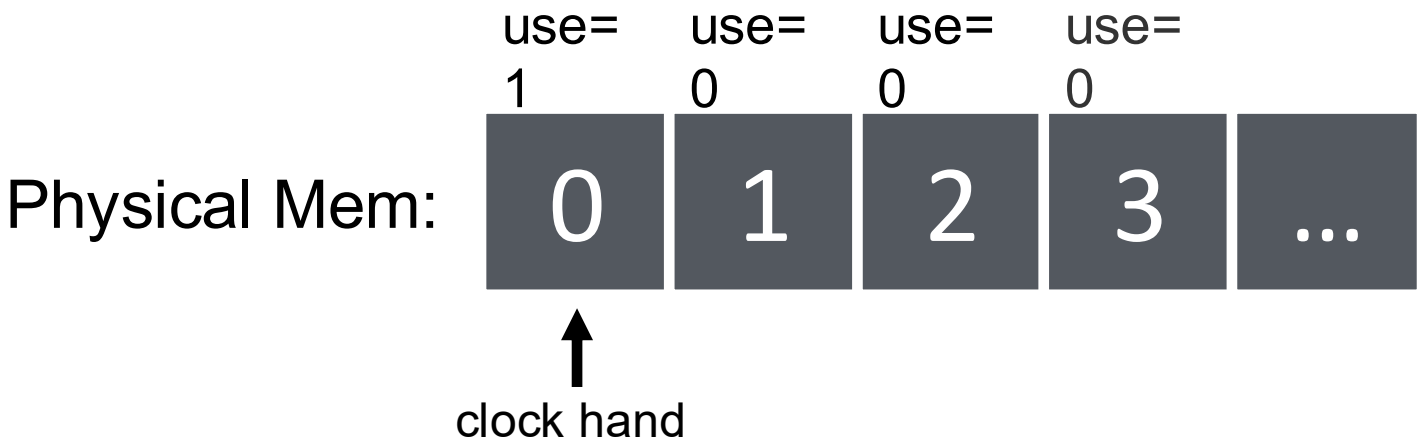
Clock:
Look For a Page



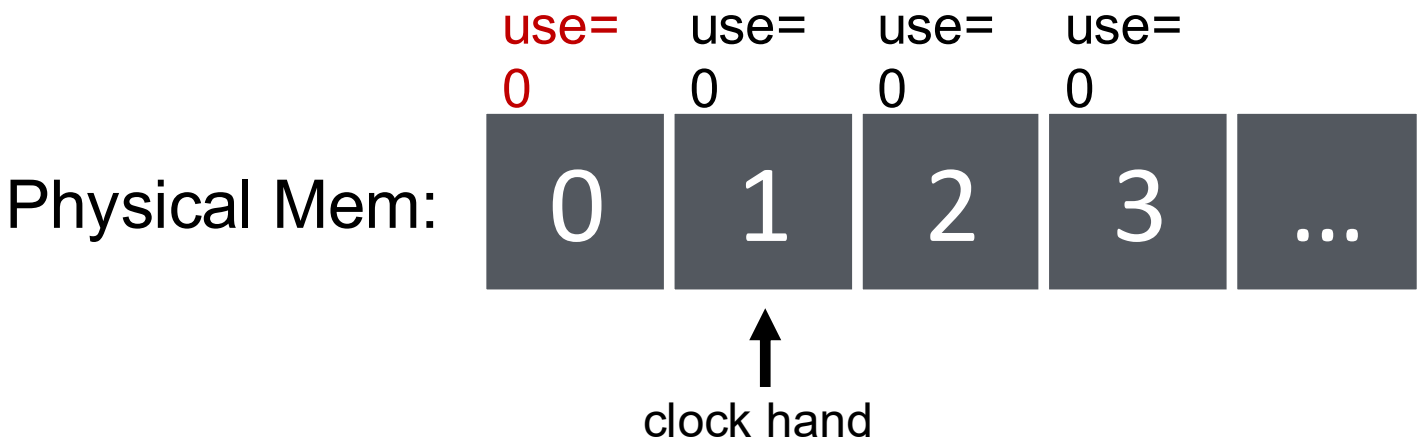
Clock:
Look For a Page



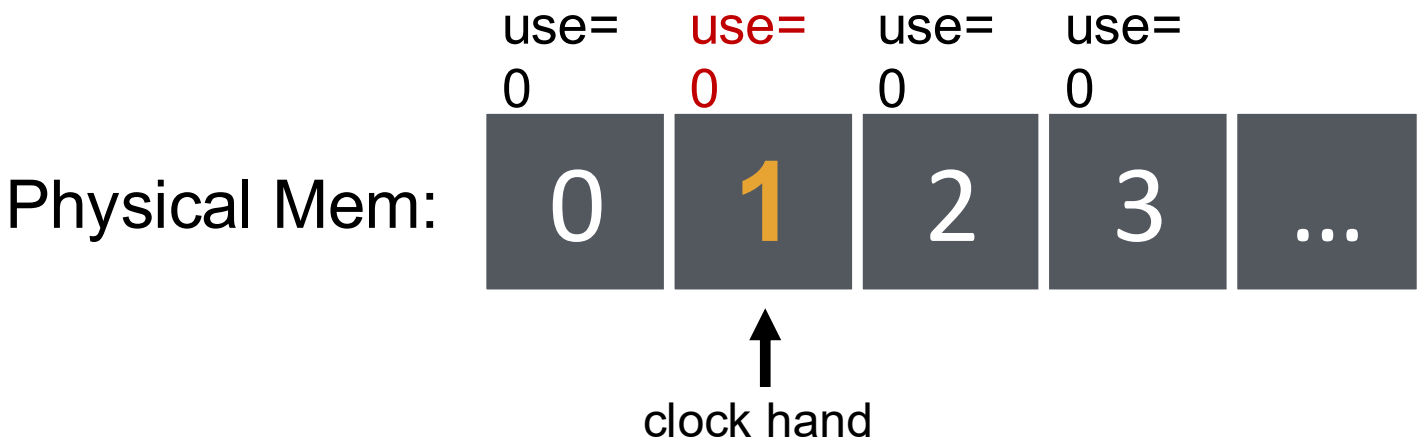
Clock:
Look For a Page



Clock:
Look For a Page



Clock:
Look For a Page



evict **page 1** because it has not been recently used

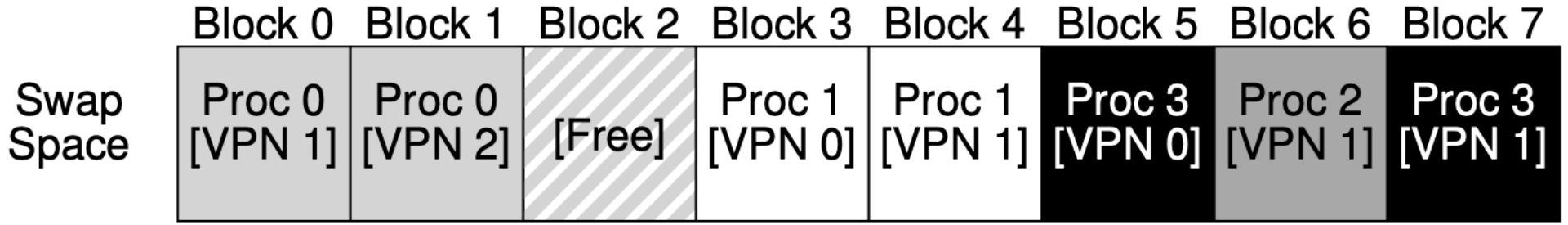
Clock Extensions

Use modified ("dirty") bit to prefer to retain modified pages in memory

- Intuition: More expensive to replace dirty pages
  - Modified pages must be written to disk, clean pages can be simply replaced
- First replace pages that have `use` bit and `modified` bit cleared

Replace multiple pages at once

- Intuition: Expensive to run replacement algorithm only to write a single block to disk; maybe we'll need to write to disk soon again?
- Batching: Find multiple victims in each run and maintain a free list



Clock Extensions

Add software counter (“chance”) to track use frequency

- Intuition: Want to differentiate pages by how much they are accessed
- Increment software counter if `use` bit is 0
- Accessing a page sets the `use` bit
- Replace when chance exceeds some specified limit

What if no hardware support?

What can the OS do if hardware does not help maintain a `use` bit (or `dirty` bit)?

- Can the OS "emulate" these bits?

- Can the OS get control (i.e., generate a trap) every time the `use` bit should be set? i.e., when the page is accessed?

Conclusion

Illusion of virtual memory: Processes can run when the sum of virtual address spaces is larger than physical memory

Mechanism:

- Extend page table entry with “present” bit
- OS handles page faults (or page misses) by reading in the desired page from disk

Policy:

- Page selection – demand paging, prefetching, hints
- Page replacement – OPT, FIFO, LRU, others

Implementations (clock) approximate LRU